

EL Calls

Developer Guide — Architecture, Data Model, Edge Functions, Performance and Security

Version 2.6 • June 2026

1. Stack Overview

Frontend: React 18, Vite 5, TypeScript 5, Tailwind v3, shadcn/ui, TanStack Query, React Router v6.
Backend: Lovable Cloud (managed Supabase) — Postgres 15 with RLS, Deno edge functions, Realtime broadcast, Storage. Voice: Twilio Programmable Voice + browser WebRTC. AI: Lovable AI Gateway (Gemini 2.5 Flash for chat/officer-chat, Whisper-equivalent for transcription).

1.1 Repository layout

```
src/  
components/ shadcn-based UI, feature panels under dashboard/  
hooks/ useAuth, useUserRole, useWebRTCcall, useAgentPresence...  
integrations/supabase/ generated client + types (DO NOT EDIT)  
pages/ route components  
lib/ phone, normalizers, utilities  
supabase/  
functions/ Deno edge functions (one folder = one function)  
migrations/ timestamped SQL files  
load-tests/ k6 scripts and reports  
public/docs/ User and Developer PDFs (this file)
```

2. Routing and Navigation

React Router v6 with a ScrollToTop component remounting on every pathname change. AdminRoute and ProtectedRoute wrap restricted routes and call useUserRole, which queries user_roles via the has_role RPC. Sidebar items are filtered by role on the client; server enforcement is the source of truth.

3. Data Model

3.1 Core tables

Table	Purpose
profiles	Public user info, linked to auth.users.
user_roles	Role assignment (admin/supervisor/agent). Separate table to prevent escalation.
clients	Customer master records with tags.

Table	Purpose
call_campaigns / campaign_types	Campaign definitions and templates.
campaign_runs / campaign_jobs	Per-execution and per-target row.
outbound_calls	One row per dialed call. 6 composite indexes added in 2026-06 migration.
outbound_call_events (+monthly partitions)	Append-only event log from Twilio status callbacks.
call_queue / call_transfers	Live routing primitives.
agent_status / agent_skills / agent_shifts	Presence, capabilities, rota.
escalation_settings / emergency_alerts	Escalation rules and raised alerts.
call_transcriptions / campaign_recordings	ASR output and recording metadata.
ivr_menu_options / supported_languages	IVR config and localisation.
audit_log / error_events / system_health_metrics	Observability.

3.2 Row Level Security

Every public table is RLS-enabled. The default pattern: enable RLS, GRANT to the appropriate roles, then create policies that delegate to security-definer helpers.

```
create or replace function public.has_role(_uid uuid, _role app_role)
returns boolean language sql stable security definer
set search_path = public as $$
select exists(select 1 from public.user_roles where user_id=_uid and role=_role)
$$;

create policy "Admins manage all" on public.call_campaigns
for all to authenticated using (public.has_role(auth.uid(), 'admin'));
```

Partitioned tables (outbound_call_events_YYYY_MM) inherit RLS from the parent and are explicitly RLS-enabled on creation by create_outbound_call_events_partition().

4. Edge Functions

All functions live under *supabase/functions/<name>*. Naming convention: kebab-case folder; index.ts as endpoint. Shared utilities in *_shared/* include twilio-verify (signature validation), idempotency, rate-limit, audit, phone and monitor.

Function	Auth	Purpose
ai-voice-call	Twilio signature	Initial TwiML for outbound dialer.
ai-voice-call-status	Twilio signature	Inserts every Twilio status into <code>outbound_call_events</code> (pure append, no UPDATE).
ai-voice-call-dtmf*	Twilio signature	DTMF gather flows including appointment + language.
ai-voice-call-bridge-ivr	Twilio signature	Connects callers to the configured IVR menu.
process-call-events	CRON_SECRET	Drains <code>outbound_call_events</code> into <code>outbound_calls</code> in batches of 500.
campaign-scheduler	Staff JWT	Plans which campaigns to run within dialing windows.
campaign-enqueue	Staff JWT	Enqueues <code>campaign_jobs</code> into <code>call_queue</code> .
campaign-worker	CRON_SECRET	Pops jobs and triggers Twilio dials.
campaign-control	Staff JWT	Start/pause/resume campaigns.
retry-campaign-calls	CRON_SECRET or staff	Re-attempts failed jobs per retry policy.
reconcile-call-status	CRON_SECRET	Hourly reconciliation against Twilio API.
smart-call-routing	Staff JWT	Scores agents by skills+proficiency+performance.
analyze-call-sentiment	Internal	Lovable AI sentiment per transcript turn.
transcribe-call / voice-to-text	Staff JWT, 10MB cap	Whisper transcription.
officer-chat / ai-policy-notes	Authenticated	Gemini 2.5 Flash assistants grounded in <code>knowledge_base</code> .
send-agent-invitation	Staff JWT	Server forces <code>invitedBy</code> = caller identity.
send-escalation-notification	Staff JWT	Pulls recipients from <code>escalation_settings</code> only.
send-appointment-email / send-payment-email	Internal	Transactional emails via SendGrid.
manage-agents / manage-app-secrets	Admin JWT	Admin CRUD.
twilio-recent-calls / twilio-verify	Staff JWT	Diagnostics.

Function	Auth	Purpose
suggest-tag	Staff JWT	Heuristic + AI tag suggestions.
sample-data	Staff JWT	Seed demo data.
gdpr-export / gdpr-delete	Admin JWT	Privacy operations.
realtime-chat	Authenticated	WebSocket relay for live chat panel.

4.1 Error handling contract

Edge functions log full errors server-side and return a generic message to the client to avoid information disclosure:

```
try { /* work */ }
catch (e) {
  console.error('fn-name failed', e);
  return new Response(JSON.stringify({ error: 'An unexpected error occurred' })),
  { status: 500, headers: corsJson });
}
```

4.2 Twilio webhook security

Inbound Twilio callbacks (status, DTMF, IVR bridge) validate the X-Twilio-Signature header via `_shared/twilio-verify.ts` using the auth-token secret. Replay protection uses the idempotency helper keyed on CallSid+CallStatus.

5. Performance Architecture

5.1 Index strategy

Migration 2026-06-22 added six composite indexes on `outbound_calls`:

- `twilio_call_sid` (webhook lookups, O(1))
- `campaign_id + created_at DESC` (campaign dashboards)
- `call_status + created_at DESC` (live queues)
- `agent_email + created_at DESC` (agent performance)
- `phone_number` (fallback matching)
- `client_id + created_at DESC` (client-level queries)

5.2 Event-log + partitioning

`ai-voice-call-status` is hot during campaigns: 4-6 callbacks per call. The previous design did an `UPDATE` on `outbound_calls` per callback and contended on the same row. The new design:

- Webhook does pure `INSERT` into `outbound_call_events` (range-partitioned by month).
- `process-call-events` runs every 5-10s on a cron, batching 500 rows.
- It resolves call IDs via `twilio_call_sid` or `parent_sid` index, then applies the final state with a single `UPDATE` per call inside one transaction.
- `create_outbound_call_events_partition()` pre-creates next month's partition and enables RLS automatically.

```
-- drainer (simplified)
select public.process_outbound_call_events(500);
```

5.3 Load test results (k6)

Scenario	Load	p95	Throughput	Notes
Campaign execution	200 RPS, 5m	220 ms	98.4% OK	DB CPU 55%.
Concurrent dialing	500 active calls	n/a	Stable	Twilio rate is the cap.
Webhook processing	1200 RPS	85 ms	100% OK (post-fix)	Was 22% errors before event-log.
Dashboard traffic	200 vUsers, 10m	180 ms	99.9% OK	Indexes cut p95 by 6x.

Max sustained capacity: ~130k calls/day per project on Lovable Cloud Pro plan; bottleneck shifts to Twilio concurrent-call quota above that.

6. Security

- Roles stored in `user_roles` table only; checked via `has_role` security-definer.

- RLS on every public table and every partition. GRANTS explicit, never relying on defaults.
- Edge functions classify auth: public + Twilio-signed, authenticated, staff JWT, admin JWT, CRON_SECRET.
- No SUPABASE_SERVICE_ROLE_KEY usage on the client; only edge functions read it.
- Generic error messages on all public surfaces; full errors logged via console.error.
- Rate-limiting helper in `_shared/rate-limit.ts` protects expensive endpoints.
- Audit-log entries written for all role changes, GDPR ops, secret reads, escalations.
- Auth redirects pinned to `https://elcalls.lovable.app` to avoid open-redirect.
- Sign-out forces redirect to `'/'` and clears all client state.

7. Realtime and Presence

Supabase Realtime broadcast is used for: live transcription updates, queue and transfer changes, supervisor live view, and officer-chat streaming. `useAgentPresence` sends a heartbeat every 30s and flips users to *away* after 5 minutes of inactivity. WebRTC signalling rides the same Realtime channel.

8. WebRTC Calling

`useWebRTCCall` sets up `RTCPeerConnection` per call, with ICE servers fetched from a signed edge function. SDP offers/answers and ICE candidates broadcast through a Supabase Realtime channel scoped to the call ID. The LiveKit credentials slot exists for a future migration to LiveKit SFU when concurrent browser-to-browser calls exceed the mesh limit.

9. AI Integrations

- Officer Chat and AI Policy Notes use Lovable AI Gateway with google/gemini-2.5-flash by default.
- Tag suggestions use a heuristic pass plus an AI fallback.
- Sentiment runs per transcript turn and writes to `call_transcriptions.sentiment`.
- Transcription uses Whisper via voice-to-text/transcribe-call; 10MB payload cap.

All AI calls go through the gateway — no third-party API keys are required from the customer for chat/embeddings/STT/TTS unless they want a specific provider.

10. Local Development

```
bun install
bun run dev # vite on :8080
bunx vitest run # unit tests
bunx tsgo --noEmit # typecheck (preferred over tsc)

# edge functions: deploy via Lovable Cloud, logs in dashboard
# never edit src/integrations/supabase/{client,types}.ts (auto-gen)
```

11. Migrations

Every CREATE TABLE in *public* is followed in the same migration by GRANTs and RLS enable + policies. The repo includes a migration that backfills RLS on existing partitions and adds the `create_outbound_call_events_partition()` helper.

12. Observability

- `error_events` captures handled exceptions from edge functions.
- `system_health_metrics` records throughput, queue depth, ASR latency.
- `audit_log` records every sensitive action (role changes, secret reads, GDPR ops).
- Lovable Cloud function logs are the primary surface for live debugging.

13. Testing

- Unit tests: vitest, including `src/lib/phone.test.ts` and `supabase/functions/_shared/*.test.ts`.
- Load tests: k6 scripts under `load-tests/scripts` (01 campaign, 02 dialing, 03 webhook, 04 dashboard).
- Run all: `bunx vitest run`; load report at `load-tests/reports/REPORT.md`.

14. Deployment

Use the Publish action in Lovable. Custom domains map to the published URL. Cron schedules for process-call-events, campaign-worker, reconcile-call-status and retry-campaign-calls are configured in Lovable Cloud. CRON_SECRET protects all cron-triggered endpoints.

15. Extending the Platform

- New campaign type: insert into campaign_types, add any IVR menu options, deploy.
- New language: insert supported_languages row, upload audio via LanguageAudioUploader, add translation strings.
- New role: extend app_role enum, add policies, update AdminRoute logic.
- New AI feature: prefer Lovable AI Gateway; never embed third-party keys in client.

16. Coding Conventions

- Tailwind tokens only; no hardcoded colour utilities (no text-white, bg-[#hex]).
- Long forms broken into tabs so action buttons stay visible.
- Wow aesthetic: gradients, glassmorphism, micro-interactions (ripples, confetti).
- Officer chat always rendered as a floating non-modal right-side panel.
- Sidebar collapsible from the left; scroll to top on every nav.

17. Appendix: Useful RPCs

RPC	Returns	Used for
has_role(uid, role)	boolean	RLS policies
is_admin() / is_staff() / is_supervisor_or_admin()	boolean	Edge function gating
process_outbound_call_events(limit)	integer	Drainer cron
create_outbound_call_events_partition(date)	void	Monthly partition creation

18. Active Directory (SAML SSO) Setup

The Active Directory configurator (Setup ! People ! Active Directory) provides a guided, auto-validated workflow for connecting EL Calls to a corporate IdP. Implementation: `src/components/dashboard/ActiveDirectorySetup.tsx`.

18.1 Supported IdP Types

A discriminated `idpTemplates` map drives the dynamic field set per directory type: Microsoft Entra ID (Azure AD), AD FS, Okta, OneLogin, Google Workspace and Generic SAML 2.0. Selecting a type swaps the required fields and resets verification state.

18.2 Draft Save & Resume

Configuration is persisted as a draft in `localStorage` under key `el:ad-setup:draft:v1`. The draft preserves: selected IdP type, captured field values, allowed domains, pilot users, default role, enforcement toggles, break-glass email, and the verified metadata snapshot (when status is 'ok'). On dialog open the draft is hydrated and the user is notified. A 'Draft saved' badge plus a 'Resume Active Directory setup' CTA appear on the card when a draft exists. Submission clears the draft via `clearDraft()`.

18.3 Entra ID Consistency Validation

When the IdP type is `entra`, an `entraIssues` memo runs the following checks before any checklist item turns green:

- Tenant ID matches GUID regex `^[0-9a-f]{8}-...-[0-9a-f]{12}$`
- Enterprise App Object ID matches the same GUID regex
- Federation Metadata URL host is `login.microsoftonline.com` (or `.us`)
- Metadata URL contains the supplied Tenant ID
- Metadata URL path includes `/federationmetadata/`
- After fetch, the returned `entityID` references the Tenant ID

18.4 Test SSO Button

`runSsoTest()` performs a live probe against the IdP. Preconditions: all required fields supplied, metadata verified, and (for Entra) consistency checks passing. The probe fetches the SSO endpoint extracted from the verified metadata using mode: 'no-cors'; an opaque response counts as reachable. The result is stored in `ssoTest` state as `idle` | `running` | `success` | `error`, surfaced both inline and as a checklist row with a specific failure reason.

18.5 Auto-Verified Production Checklist

The checklist is computed by an `autoChecks` memo — users cannot tick items manually. Each entry exposes `label`, `done`, and a `why` string explaining the current state. Green rows show a confirmation; red rows show the precise blocker.

- All required IdP fields supplied — lists missing field labels when red.
- IdP details consistent — surfaces the first Entra consistency error.
- IdP metadata verified & reachable — reports HTTP/parse failure or asks for Verify click.
- Live SSO test passed — shows the probed hostname or the specific failure reason.
- Email domains allowlisted — lists malformed domains.
- Pilot users supplied (2-5 valid emails) — explains count or format violation.
- Single Logout + SSO enforcement enabled.
- Break-glass admin email captured (or explicitly disabled).

Submit is enabled only when every check is green. On submit the component dispatches a `lovable:saml-sso-submit` `CustomEvent` with the full payload and clears the draft. Server-side SAML trust finalisation is handled by Lovable Cloud SSO configuration.

18.6 Service Provider Constants

- Entity ID: `https://elcalls.lovable.app/saml/metadata`
- ACS / Reply URL: `https://elcalls.lovable.app/saml/acs`
- Single Logout URL: `https://elcalls.lovable.app/saml/slo`
- NameID format: `emailAddress`